

```

# =====
# BOOLEAN ALGEBRA & ADDER CIRCUITS - GOOGLE COLAB VERSION
# =====

import ipywidgets as widgets
from IPython.display import display, clear_output
import matplotlib.pyplot as plt
from matplotlib.patches import Arc, Polygon, Circle

# =====
# INPUT VALIDATION
# =====

def get_boolean(value, name):

    value = str(value).strip()

    if value not in ("0", "1"):
        print(f"❌ Invalid Input: {name} must be either 0 or 1.")
        return None

    return int(value)

# =====
# DRAWING FUNCTIONS
# =====

def draw_and_gate(ax, x, y, label="AND"):

    # AND gate body
    ax.plot(
        [x, x + 0.8],
        [y - 0.5, y - 0.5],
        'k'
    )

    ax.plot(
        [x, x + 0.8],
        [y + 0.5, y + 0.5],
        'k'
    )

    ax.plot(
        [x, x],
        [y - 0.5, y + 0.5],
        'k'
    )

    arc = Arc(
        (x + 0.8, y),
        1.0,
        1.0,
        theta1=-90,
        theta2=90,
        color='black'
    )

    ax.add_patch(arc)

    ax.text(
        x + 0.35,
        y,
        label,
        ha='center',
        va='center',
        fontsize=9,
        fontweight='bold'
    )

    return x + 1.3

def draw_or_gate(ax, x, y, label="OR"):

    points = [
        (x, y - 0.5),
        (x + 0.5, y - 0.45),
        (x + 1.2, y),
        (x + 0.5, y + 0.45),
        (x, y + 0.5)
    ]

```

```

    (x + 0.5, y + 0.5),
    (x, y + 0.5)
]

polygon = Polygon(
    points,
    closed=False,
    fill=False,
    edgecolor='black'
)

ax.add_patch(polygon)

ax.text(
    x + 0.5,
    y,
    label,
    ha='center',
    va='center',
    fontsize=9,
    fontweight='bold'
)

return x + 1.2

def draw_xor_gate(ax, x, y, label="XOR"):

    end = draw_or_gate(ax, x + 0.1, y, label)

    # Extra XOR curve
    arc = Arc(
        (x + 0.05, y),
        0.5,
        1.0,
        theta1=-90,
        theta2=90,
        color='black'
    )

    ax.add_patch(arc)

    return end + 0.1

def draw_not_gate(ax, x, y):

    triangle = Polygon(
        [
            (x, y - 0.35),
            (x, y + 0.35),
            (x + 0.8, y)
        ],
        closed=True,
        fill=False,
        edgecolor='black'
    )

    ax.add_patch(triangle)

    ax.text(
        x + 0.3,
        y,
        "NOT",
        ha='center',
        va='center',
        fontsize=9,
        fontweight='bold'
    )

    # NOT bubble
    circle = Circle(
        (x + 0.88, y),
        0.08,
        fill=False
    )

    ax.add_patch(circle)

    return x + 0.96

def draw_nand_gate(ax, x, y):

```

```

    end = draw_and_gate(ax, x, y, "AND")

    circle = Circle(
        (end, y),
        0.08,
        fill=False
    )

    ax.add_patch(circle)

    return end + 0.1

def draw_nor_gate(ax, x, y):

    end = draw_or_gate(ax, x, y, "OR")

    circle = Circle(
        (end, y),
        0.08,
        fill=False
    )

    ax.add_patch(circle)

    return end + 0.1

# =====
# BASIC CIRCUIT
# =====

def draw_basic_circuit(operation):

    fig, ax = plt.subplots(figsize=(10, 4))

    ax.set_xlim(0, 10)
    ax.set_ylim(0, 5)

    ax.axis("off")

    ax.set_title(
        operation + " GATE",
        fontsize=16,
        fontweight="bold"
    )

    if operation == "NOT":

        ax.text(
            0.5, 2.5,
            "A",
            fontsize=12,
            fontweight="bold"
        )

        ax.plot(
            [0.7, 2.5],
            [2.5, 2.5],
            'k'
        )

        end = draw_not_gate(
            ax,
            2.5,
            2.5
        )

        ax.plot(
            [end, 7],
            [2.5, 2.5],
            'k'
        )

        ax.text(
            7.2, 2.5,
            "V",
            fontsize=12,
            fontweight="bold"
        )

    else:

```

```

ax.text(
    0.5, 3.2,
    "A",
    fontsize=12,
    fontweight="bold"
)

ax.text(
    0.5, 1.8,
    "B",
    fontsize=12,
    fontweight="bold"
)

# Input wires
ax.plot(
    [0.7, 3],
    [3.2, 3.2],
    'k'
)

ax.plot(
    [0.7, 3],
    [1.8, 1.8],
    'k'
)

gate_x = 3
gate_y = 2.5

if operation == "AND":

    end = draw_and_gate(
        ax,
        gate_x,
        gate_y
    )

elif operation == "OR":

    end = draw_or_gate(
        ax,
        gate_x,
        gate_y
    )

elif operation == "NAND":

    end = draw_nand_gate(
        ax,
        gate_x,
        gate_y
    )

elif operation == "NOR":

    end = draw_nor_gate(
        ax,
        gate_x,
        gate_y
    )

elif operation == "XOR":

    end = draw_xor_gate(
        ax,
        gate_x,
        gate_y
    )

elif operation == "XNOR":

    end = draw_xor_gate(
        ax,
        gate_x,
        gate_y
    )

    circle = Circle(
        (end, gate_y),
        0.08,
        fill=False

```

```

    )

    ax.add_patch(circle)

# Output
ax.plot(
    [end, 7],
    [gate_y, gate_y],
    'k'
)

ax.text(
    7.2,
    gate_y,
    "y",
    fontsize=12,
    fontweight="bold"
)

plt.show()

# =====
# HALF ADDER
# =====

def draw_half_adder():

    fig, ax = plt.subplots(figsize=(11, 5))

    ax.set_xlim(0, 11)
    ax.set_ylim(0, 6)

    ax.axis("off")

    ax.set_title(
        "HALF ADDER",
        fontsize=16,
        fontweight="bold"
    )

    # Inputs
    ax.text(
        0.5, 4.5,
        "A",
        fontsize=12,
        fontweight="bold"
    )

    ax.text(
        0.5, 1.5,
        "B",
        fontsize=12,
        fontweight="bold"
    )

    # XOR gate
    draw_xor_gate(
        ax,
        4,
        4
    )

    # A to XOR
    ax.plot(
        [0.8, 4],
        [4.5, 4.5],
        'k'
    )

    # B to XOR
    ax.plot(
        [0.8, 2.5],
        [1.5, 1.5],
        'k'
    )

    ax.plot(
        [2.5, 2.5],
        [1.5, 4],
        'k'
    )

```

```
ax.plot(
    [2.5, 4],
    [4, 4],
    'k'
)

# XOR output
ax.plot(
    [5.3, 8],
    [4, 4],
    'k'
)

ax.text(
    8.2, 4,
    "SUM",
    fontsize=12,
    fontweight="bold"
)

# AND gate
draw_and_gate(
    ax,
    4,
    1.5,
    "AND"
)

# A to AND
ax.plot(
    [0.8, 2],
    [4.5, 4.5],
    'k'
)

ax.plot(
    [2, 2],
    [4.5, 2],
    'k'
)

ax.plot(
    [2, 4],
    [2, 2],
    'k'
)

# B to AND
ax.plot(
    [0.8, 1.5],
    [1.5, 1.5],
    'k'
)

ax.plot(
    [1.5, 1.5],
    [1.5, 1],
    'k'
)

ax.plot(
    [1.5, 4],
    [1, 1],
    'k'
)

# Carry
ax.plot(
    [5.3, 8],
    [1.5, 1.5],
    'k'
)

ax.text(
    8.2, 1.5,
    "CARRY",
    fontsize=12,
    fontweight="bold"
)

plt.show()
```

```
# =====
# FULL ADDER
# =====

def draw_full_adder():

    fig, ax = plt.subplots(figsize=(12, 6))

    ax.set_xlim(0, 12)
    ax.set_ylim(0, 7)

    ax.axis("off")

    ax.set_title(
        "FULL ADDER",
        fontsize=16,
        fontweight="bold"
    )

    # Inputs
    ax.text(
        0.3, 5.5,
        "A",
        fontsize=12,
        fontweight="bold"
    )

    ax.text(
        0.3, 4.5,
        "B",
        fontsize=12,
        fontweight="bold"
    )

    ax.text(
        3.5, 3.3,
        "Cin",
        fontsize=12,
        fontweight="bold"
    )

    # First XOR
    draw_xor_gate(
        ax,
        2,
        5
    )

    # A
    ax.plot(
        [0.6, 2],
        [5.5, 5.5],
        'k'
    )

    # B
    ax.plot(
        [0.6, 1.2],
        [4.5, 4.5],
        'k'
    )

    ax.plot(
        [1.2, 1.2],
        [4.5, 5],
        'k'
    )

    ax.plot(
        [1.2, 2],
        [5, 5],
        'k'
    )

    # Second XOR
    draw_xor_gate(
        ax,
        5.5,
        5
    )

    # First XOR output
```

```
ax.plot(
    [3.3, 5.5],
    [5, 5],
    'k'
)

# Cin
ax.plot(
    [4, 4],
    [3.3, 5],
    'k'
)

ax.plot(
    [4, 5.5],
    [5, 5],
    'k'
)

# SUM
ax.plot(
    [6.8, 9],
    [5, 5],
    'k'
)

ax.text(
    9.2, 5,
    "SUM",
    fontsize=12,
    fontweight="bold"
)

# First AND
draw_and_gate(
    ax,
    2,
    2,
    "AND"
)

# A to AND
ax.plot(
    [0.6, 1],
    [5.5, 5.5],
    'k'
)

ax.plot(
    [1, 1],
    [5.5, 2.5],
    'k'
)

ax.plot(
    [1, 2],
    [2.5, 2.5],
    'k'
)

# B to AND
ax.plot(
    [0.6, 1.3],
    [4.5, 4.5],
    'k'
)

ax.plot(
    [1.3, 1.3],
    [4.5, 1.5],
    'k'
)

ax.plot(
    [1.3, 2],
    [1.5, 1.5],
    'k'
)

# Second AND
draw_and_gate(
    ax,
```

```
    5.5,  
    2,  
    "AND"  
)  
  
# XOR output  
ax.plot(  
    [3.3, 4.5],  
    [5, 5],  
    'k'  
)  
  
ax.plot(  
    [4.5, 4.5],  
    [5, 2.5],  
    'k'  
)  
  
ax.plot(  
    [4.5, 5.5],  
    [2.5, 2.5],  
    'k'  
)  
  
# Cin  
ax.plot(  
    [4, 4],  
    [3.3, 1.5],  
    'k'  
)  
  
ax.plot(  
    [4, 5.5],  
    [1.5, 1.5],  
    'k'  
)  
  
# OR gate  
draw_or_gate(  
    ax,  
    8,  
    2,  
    "OR"  
)  
  
# First AND output  
ax.plot(  
    [3.3, 7],  
    [2, 2],  
    'k'  
)  
  
ax.plot(  
    [7, 8],  
    [2, 2.3],  
    'k'  
)  
  
# Second AND output  
ax.plot(  
    [6.8, 7.2],  
    [2, 2],  
    'k'  
)  
  
ax.plot(  
    [7.2, 7.2],  
    [2, 1.7],  
    'k'  
)  
  
ax.plot(  
    [7.2, 8],  
    [1.7, 1.7],  
    'k'  
)  
  
# Carry  
ax.plot(  
    [9.2, 10.5],  
    [2, 2],  
    'k'  
)  
)
```

```

ax.text(
    10.6, 2,
    "CARRY",
    fontsize=12,
    fontweight="bold"
)

plt.show()

# =====
# CALCULATION
# =====

def calculate(operation, A_value, B_value, Cin_value):

    A = get_boolean(A_value, "A")

    if A is None:
        return

    # -----
    # BASIC OPERATIONS
    # -----

    if operation in [
        "AND",
        "OR",
        "NAND",
        "NOR",
        "XOR",
        "XNOR"
    ]:

        B = get_boolean(B_value, "B")

        if B is None:
            return

        if operation == "AND":
            result = A & B

        elif operation == "OR":
            result = A | B

        elif operation == "NAND":
            result = int(not (A & B))

        elif operation == "NOR":
            result = int(not (A | B))

        elif operation == "XOR":
            result = A ^ B

        elif operation == "XNOR":
            result = int(not (A ^ B))

        return f"Output Y = {result}"

    # -----
    # NOT
    # -----

    elif operation == "NOT":

        result = int(not A)

        return f"Output Y = {result}"

    # -----
    # HALF ADDER
    # -----

    elif operation == "Half Adder":

        B = get_boolean(B_value, "B")

        if B is None:
            return

        Sum = A ^ B
        Carry = A & B

```

```

        return f"SUM = {Sum}      CARRY = {Carry}"

# -----
# FULL ADDER
# -----

elif operation == "Full Adder":

    B = get_boolean(B_value, "B")
    Cin = get_boolean(Cin_value, "Cin")

    if B is None or Cin is None:
        return

    Sum = A ^ B ^ Cin

    Carry = (A & B) | (Cin & (A ^ B))

    return f"SUM = {Sum}      CARRY = {Carry}"

# =====
# OPERATION CHANGE
# =====

def operation_changed(change):

    operation = change["new"]

    # Clear output
    output_area.clear_output()

    # Draw selected circuit
    with circuit_area:

        clear_output(wait=True)

        if operation == "Half Adder":

            draw_half_adder()

        elif operation == "Full Adder":

            draw_full_adder()

        else:

            draw_basic_circuit(operation)

# =====
# CALCULATE BUTTON
# =====

def calculate_button_clicked(button):

    with output_area:

        clear_output()

        result = calculate(
            operation_dropdown.value,
            A_input.value,
            B_input.value,
            Cin_input.value
        )

        if result is not None:
            print("✅", result)

# =====
# CLEAR / EXIT
# =====

def exit_program(button):

    with output_area:
        clear_output()

        print("Program terminated.")

```

```

with circuit_area:
    clear_output()

# =====
# WIDGETS
# =====

operation_dropdown = widgets.Dropdown(

    options=[
        "AND",
        "OR",
        "NOT",
        "NAND",
        "NOR",
        "XOR",
        "XNOR",
        "Half Adder",
        "Full Adder"
    ],

    value="AND",

    description="Operation:"
)

A_input = widgets.Text(
    value="",
    description="A:"
)

B_input = widgets.Text(
    value="",
    description="B:"
)

Cin_input = widgets.Text(
    value="",
    description="Cin:"
)

calculate_button = widgets.Button(
    description="Calculate"
)

exit_button = widgets.Button(
    description="Exit"
)

output_area = widgets.Output()

circuit_area = widgets.Output()

# =====
# BUTTON EVENTS
# =====

operation_dropdown.observe(
    operation_changed,
    names="value"
)

calculate_button.on_click(
    calculate_button_clicked
)

exit_button.on_click(
    exit_program
)

# =====
# DISPLAY PROGRAM
# =====

```

```

title = widgets.HTML(
    "<h2>BOOLEAN ALGEBRA & ADDER CIRCUITS</h2>"
)

display(title)

display(operation_dropdown)

display(
    widgets.HBox([
        A_input,
        B_input,
        Cin_input
    ])
)

display(
    widgets.HBox([
        calculate_button,
        exit_button
    ])
)

display(output_area)

display(circuit_area)

# =====
# INITIAL CIRCUIT
# =====

with circuit_area:
    draw_basic_circuit("AND")

```

BOOLEAN ALGEBRA & ADDER CIRCUITS

Operation:

A:

B:

Cin:

✓ SUM = 0 CARRY = 1

FULL ADDER



